

Phase 4: Add knowledge and retrieval

1. Objective

This document provides the complete implementation for **Phase 4: Add Knowledge & Retrieval** for the project. In earlier phases:

- Phase 1 defined the banking workflow, constraints, and user personas.
- Phase 2 created a rule-based banking assistant.
- Phase 3 upgraded the system using LLM reasoning and safety-aware prompts.
- Phase 4 improves the system further using Retrieval-Augmented Generation (RAG), semantic search, embeddings, and vector databases so the agent can answer banking questions using trusted internal banking knowledge.

For Phase 4, we will build on our Phase 3 agent and add:

- Embeddings
- ChromaDB vector store
- Semantic search
- RAG prompt
- Comparison between "LLM only" and "LLM + Retrieval"
- Missing information handling

2. Architecture:

Customer Query -> Embedding -> ChromaDB -> Relevant Documents -> Prompt -> GPT

-> Answer

Some points here about the RAG:

-> An embedding converts the sentence into a list of numbers (a vector).

-> ChromaDB is simply a **vector database**.

Instead of storing documents like:

Document A

Document B

Document C

it stores:

Embedding A → Document A

Embedding B → Document B

Embedding C → Document C

Suppose your database contains:

Doc 1: Leave Policy

Doc 2: Expense Policy

Doc 3: Travel Policy

User asks: "What is the annual leave entitlement?"

Embedding search finds:

1. Leave Policy (95% similar)
2. Travel Policy (20% similar)
3. Expense Policy (10% similar)

ChromaDB returns:

Leave Policy:

Employees receive 20 days of annual leave...

This is the retrieval step in **Retrieval-Augmented Generation (RAG)**.

Why do we need Embeddings and ChromaDB?

Without RAG: Question → GPT → Answer

3. Python code with RAG Integration

```
#Dependencies which needs to be installed
#pip install openai chromadb
```

```
import os
import re
import logging
import sqlite3
import sys
sys.modules["sqlite3"] = sys.modules.pop("pysqlite3")
import chromadb
```

```
from openai import OpenAI
```

```
# =====
# OpenAI Client
# =====
```

```
client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    base_url=os.getenv("OPENAI_BASE_URL")
)
```

```
# =====
# Logging
# =====
```

```
logging.basicConfig(
    filename="banking_agent_rag_logs.txt",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)
```

```

# =====
# ChromaDB
# =====

chroma_client = chromadb.Client()

collection = chroma_client.get_or_create_collection(
    name="banking_docs"
)

# =====
# PII Masking
# =====

def mask_pii(text):
    text = re.sub(
        r"\b\d{8,}\b",
        "[ACCOUNT_MASKED]",
        text
    )

    text = re.sub(
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
        "[EMAIL_MASKED]",
        text
    )

    return text

# =====
# Embedding Function
# =====

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )

    return response.data[0].embedding

# =====
# Load Knowledge Base
# =====

def load_documents():
    with open(
        "bank_policies.txt",
        "r",
        encoding="utf-8"
    ) as f:
        content = f.read()

    chunks = [
        chunk.strip()
        for chunk in content.split("\n\n")
        if chunk.strip()
    ]

    return chunks

# =====

```

```

# Build Vector Store
# =====

def build_vector_store():
    docs = load_documents()
    existing = collection.count()

    if existing > 0:
        return

    for idx, doc in enumerate(docs):
        embedding = get_embedding(doc)

        collection.add(
            ids=[str(idx)],
            documents=[doc],
            embeddings=[embedding]
        )

    print(
        f"Loaded {len(docs)} documents into ChromaDB"
    )

# =====
# Semantic Search
# =====

def retrieve_documents(query, top_k=3):
    query_embedding = get_embedding(query)

    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_k
    )

    docs = results["documents"][0]

    return docs

# =====
# LLM WITHOUT Retrieval (Phase 3)
# =====

def llm_only_response(user_query):
    prompt = f"""
    You are a Banking Support Assistant.

    Answer the user's question.

    Question:
    {user_query}
    """

    response = client.responses.create(
        model="gpt-5.5",
        input=prompt
    )

    return response.output_text

# =====

```

```
# LLM WITH Retrieval (Phase 4)
```

```
# =====
```

```
def rag_response(user_query):
```

```
    retrieved_docs = retrieve_documents(  
        user_query  
    )
```

```
    if not retrieved_docs:
```

```
        return (  
            "I could not find relevant information "  
            "in the banking knowledge base."  
        )
```

```
    context = "\n\n".join(  
        retrieved_docs  
    )
```

```
    prompt = f"""
```

```
You are a Banking Support Assistant.
```

```
Use ONLY the provided context.
```

```
If the answer is not present in the context,  
say that the information is unavailable.
```

```
Context:
```

```
{context}
```

```
Question:
```

```
{user_query}
```

```
"""
```

```
    response = client.responses.create(  
        model="gpt-5.5",  
        input=prompt  
    )
```

```
    return response.output_text
```

```
# =====
```

```
# Main Application
```

```
# =====
```

```
def main():
```

```
    build_vector_store()
```

```
    print("=" * 70)
```

```
    print("Banking Support Agent – Phase 4 (RAG)")
```

```
    print("=" * 70)
```

```
    print("Type 'exit' to quit")
```

```
    while True:
```

```
        user_input = input(  
            "\nCustomer: "  
        )
```

```
        if user_input.lower() == "exit":  
            break
```

```
        try:
```

```

safe_input = mask_pii(
    user_input
)

# -----
# Without Retrieval
# -----

llm_response = (
    llm_only_response(
        user_input
    )
)

# -----
# With Retrieval
# -----

rag_answer = (
    rag_response(
        user_input
    )
)

# -----
# Logging
# -----

logging.info(
    f"""
Customer={safe_input}

WITHOUT RETRIEVAL:
{llm_response}

WITH RETRIEVAL:
{rag_answer}

-----
"""

)

# -----
# Display Comparison
# -----

print("\n" + "=" * 70)
print("WITHOUT RETRIEVAL (Phase 3)")
print("=" * 70)

print(llm_response)

print("\n" + "=" * 70)
print("WITH RETRIEVAL (Phase 4)")
print("=" * 70)

print(rag_answer)

except Exception as e:

    print(
        f"\nAgent Error: {e}"
    )

    logging.error(
        f"Error: {e}"
    )

```

```

    )

if __name__ == "__main__":
    main()

```

4. Comparison between with and without retrieval

We can clearly observe the difference in the screenshot below between the responses generated with and without RAG. The RAG-based system retrieves the relevant information stored in the documents and uses it to generate an accurate response.

Customer=What is the current home loan interest rate?

WITHOUT RETRIEVAL:
Home loan interest rates can change frequently and depend on factors like the loan type, amount, term, deposit/equity, and your credit profile.

For the most current home loan interest rate, please check the bank's official website, mobile app, or contact a home loan specialist. If you tell me the type of home loan you're interested in—fixed, variable, first-home buyer, refinance, or investment—I can help explain what to look for.

WITH RETRIEVAL:
The current home loan interest rate is ****8.25% per annum****.

2026-05-31 16:02:58,427 - HTTP Request: POST https://openai.vocareum.com/v1/responses "HTTP/1.1 200 OK"
 2026-05-31 16:02:58,982 - HTTP Request: POST https://openai.vocareum.com/v1/embeddings "HTTP/1.1 200 OK"
 2026-05-31 16:03:00,756 - HTTP Request: POST https://openai.vocareum.com/v1/responses "HTTP/1.1 200 OK"
 2026-05-31 16:03:00,757 -
 Customer=What are the benefits of the Travel Rewards Credit Card?

WITHOUT RETRIEVAL:
The Travel Rewards Credit Card typically offers benefits such as:

- ****Earn rewards on purchases**** — collect points or miles for everyday spending and travel purchases.
- ****Redeem for travel**** — use rewards toward flights, hotels, car rentals, cruises, or statement credits for travel.
- ****No foreign transaction fees**** — helpful when making purchases outside the country.
- ****Travel protections**** — may include trip cancellation/interruption coverage, lost luggage reimbursement, or travel accident insurance.
- ****Rental car coverage**** — secondary or primary auto rental collision coverage may be included.
- ****Fraud protection**** — zero-liability protection for unauthorized purchases.
- ****Travel perks**** — depending on the card, this may include airport lounge access, travel credits, or priority services.

Benefits can vary by card type and account terms, so it's best to review the card's benefits guide or contact customer support for the exact details.

WITH RETRIEVAL:
The Travel Rewards Card benefits are:

- 2x reward points
- Airport lounge access

5. Missing information handling

The following example demonstrates how the agent responds to a query with and without RAG. The information required to answer this query is not included in the documents provided to the RAG system.

WITHOUT RETRIEVAL:
The premium wealth management fee is ****1% of assets under management annually****.

WITH RETRIEVAL:
The information is unavailable.

6. Limitations

(1) The Banking AI Support & Advisory Agent can only retrieve information that exists in the knowledge base. If banking policies, product details, or customer support procedures are missing, outdated, or incomplete, the agent may be unable to provide a useful answer. While retrieval reduces hallucinations compared to the LLM-only version, it cannot compensate for missing source information. As a result, maintaining an accurate and up-to-date knowledge base is critical for system performance.

(2) The semantic search component retrieves documents based on embedding similarity rather than exact understanding. In some cases, the vector database may return partially relevant or irrelevant documents, especially when user queries are ambiguous or use terminology that differs from the wording in the stored documents. This can lead to incomplete responses or responses grounded in less relevant context. Additional techniques such as improved chunking, metadata filtering, reranking, and retrieval evaluation would be required for production-grade performance.

8. Conclusion

Phase 4 introduced Retrieval-Augmented Generation (RAG) capabilities into the Banking AI Support & Advisory Agent by incorporating embeddings, semantic search, and a vector database. Banking policy documents were embedded and stored in ChromaDB, enabling the agent to retrieve relevant information before generating responses. Compared to the Phase 3 LLM-only implementation, the retrieval-enhanced agent produced responses that were more grounded, consistent, and aligned with the available banking documentation.

The implementation demonstrated document preparation, embedding generation, semantic retrieval, response grounding, comparison between retrieval and non-retrieval approaches, and handling of missing information scenarios. While retrieval significantly improves response reliability, challenges related to knowledge base completeness and retrieval accuracy remain. Overall, this phase establishes a practical RAG foundation that can be further enhanced with larger document collections, improved retrieval strategies, and additional safety mechanisms in subsequent phases.